PAGE NO.....
DATE / /

5) Confidence Intervals Explained

1.) What is Confidence Interval (CI)?

→ A range of values around a sample statistic (e.g. mean) that likely contains the true population value.

→ Example: "Average income is $50K ± 3K (95% of CI)" means:
if we repeated the study 100 times, ~95 of those CIs would contain the true average.

2.) Key Components.

Term	Meaning	Example for Income Data
Point Estimate	Single best guess (e.g. sample mean)	$57,573 (mean from sample)
Margin of Error	Half the width of the CI.	± $4,000
Confidence Level	How often the CI captures the truth (e.g. 95%)	90%, 95%, 99%

→ Imagine you're guessing the average height of all students in a school. You measure 30 students and get an average of 165cm. A 95% confidence interval might say:

→ Translation: if you repeated the process 100 times, ~95 of your intervals would contain the real average.

PAGE NO.....
DATE / . /

① Point Estimates (mean / median)

→ Your single best guess from the sample.

→ Let's say Income Example is Average income is $57,573

→ Your Visualization, bull eye at $57,573

② Margin of Error: (correction).

→ How far the true value might deviate from your guess.

→ "±$4,000" means Bar extending $4,000 each way.

③ Confidence Level

→ How reliable your interval is (long-term success rate)

→ 95% confidence means 95/100 intervals contain true average.

Real-Word Analogy: weather forecasting

- Point Estimate: "Tomorrow's high will be 75°F
- Margin of Error: "±3°F" → Range: 72°F - 78°F
- Confidence Level: "90% confidence" → The meteorology's track record.

PAGE NO.....
DATE / /

Why This Matters in Data Science.

b) Avoids overconfidence: A point estimate alone hides uncertainty.

b) Supports decision-making:
→ Narrow CI (e.g. 55k - 59k) → Reliable estimate.
→ Wide CI (e.g. 40k - 70k) → Need more data.

(3) How to Calculate (Bootstrap Method).

1. Resample with replacement from your data (e.g. 1,000 times).
2. Calculate the statistic (mean, median, etc) for each resample.
3. Sort the results and trim the extremes.
   - For a 90% CI, discard the top / bottom 5% of values.
   - The remaining range is your CI.

(4) Interpretation Tips:

- Higher Confidence Level → Wider CI (e.g. 99% CI is broader than 90%)
- Larger Sample Size → Narrower CI (more data = less uncertainty).
- Not a Probability: A 95% CI doesn't mean "95% chance the true value is here". It means "95% of such intervals will capture the truth".

→ (4) b. Confidence Level Trade-off

- 90% CI: "I'm pretty sure it's between A and B"

Why This Matters in Data Science.

b) Avoids overconfidence: A point estimate alone hides uncertainty.

d) Supports decision-making:

→ Narrow CI (e.g. 55k - 59k) → Reliable estimate.

→ Wide CI (e.g. 40k - 70k) → Need more data.

③ How to Calculate (Bootstrap Method).

1. Resample with replacement from your data (e.g. 1,000 times).
2. Calculate the statistic (mean, median, etc) for each resample.
3. Sort the results and trim the extremes.
   - For a 90% CI, discard the top / bottom 5% of values.
   - The remaining range is your CI.

④ Interpretation Tips:

- Higher Confidence level → Wider CI (e.g. 99% CI is broader than 90%)
- Larger Sample Size → Narrower CI (more data = less uncertainty).
- Not a Probability: A 95% CI doesn't mean "95% chance the true value is here". It means "95% of such intervals will capture the truth".

→ ⑤ a. Confidence Level Trade-off

- 90% CI: "I'm pretty sure it's between A and B"
  → Narrower range, but 10% chance of missing.

PAGE NO.....
DATE / /

(a)
- ~~95% CI~~: I'm pretty sure it's between A and B
- 95% CI: "I'm very sure it's between C and D" → Wider range, more conservative.
- 99% CI: I'm extremely sure it's between E and F!!
  → Very wide range.

Real-world analogy

(b)
- 90% CI = Regular seatbelt.
- 95% CI = seatbelt + air bag.
- 99% CI = Full racing harness.
  → More protection (confidence) = More restriction (wider interval)

(c) Sample Size Power (The "More Data = Sharper Guess" Rule).
- n = 100: Might give CI of 50 ± 10.
- n = 1,000, Might tighten to 50 ± 3.
- n = 10,000, could reach 50 ± 1.

~~small sample~~

(d) The Biggest Misconception (what 95% Really Mean)

[X Wrong]: "There's a 95% chance the true value is in this range."

[Right]: 95% of intervals built this way contain the truth.

Thought experiment:

Imagine 100 researchers studying the same thing.

PAGE NO.....
DATE / /

- About 95 will get CIs containing the ~~value~~ true values.
- About 5 will get "unlucky" CIs that miss it → Your CI is either in the 95% or the 5% - you just don't know which!

Practical Implications for Data Science.

1) For quick decisions → Use 90% CI (accepts more it)
2) For medical trials → Use 99% CI (extremely cautious)
3) Always report sample size with CI
(n=20 vs n=2,000 tells very different stories).

(5) Common Mistakes in CI

(A) C.I vs. Prediction Interval: know the difference

(A.1) Confidence Interval (CI)

- Answers: "Where is the true average?"
- Example: "Average blood pressure is 120 ± 5 mm Hg (95% CI)"
- Visual: Target showing where the bullseye (true mean) likely is.

(A.2) Prediction Interval (PI).

- Answers: "Where will the next measurement fall?"
- Example: "Next patient's blood pressure will be 120 ± 15 mmHg (95% PI)"
- Visual: "A range covering most individual values more than averages.

PAGE NO.
DATE / /

(B) Sample Size Powers (The "More Data = Sharper Guess" Rule).

- $n \approx 100$: Might give CI of 50 ± 10
- $n = 1{,}000$ Might tighten to 50 ± 3.
- $n \approx 10{,}000$: Could reach 50 ± 1.

(2) The Normality Trap.

(A) Traditional formulas (t-distribution).

- Assume data is - normal when $n < 30$.
- Danger Zone: Skewed data like incomes, hospital stays.
- Example fail: Using t-CI for "[10k, 10M]" income data → misleading interval.

(3) Sample Size Blindness

(A) Small Samples ($n < 15$):

1) CIs become unreliable
2) Example: Drug trial with 10 patients

(B) Rule of Thumb:

- For $n < 15$: Always use bootstrap or nonparametric methods.
- For $n \geq 30$: t-distribution usually safe.

Solo Confidence Interval

purpose: Estimate population mean from a sample.
Applies when: You just want CI bounds without formal hypothesis test.
Formula: $\bar{x} \pm$ (critical value $\times SE$).

- z-test
- t-test

Pythoncode

```
# Python Example -- seeing is Believing
# This Python scripts use bootstrapping to estimate the
  90% confidence (CI) for the mean income of loan
  application.

import numpy as np

# sample data (incomes of 20 Loan Belicants)
data = np.array ([60, 55, 60, 65, 70, 45, 80, 75, 55,
                  90, 52, 58, 63, 68, 72, 48, 85,
                  77, 57, 82]).

# Bootstrap 90% of CI for the mean.
n_iterations = 1000                    → using list comprehensions
bootstrap_means = [np.mean (np.random.choice
                  (data, size = len(data), replace
                  = True))
                  for _ in range (n_iterations)]

Lower = np.percentiles (bootstrap_means, 5) # 5th
                                  percentiles
# 5th percentile.
upper = np.percentiles( bootstrap_means, 95) # 95th
                                  percentiles.

print (f" sample Mean: ${np.mean(data):.2f}${lower:.2f}
print (f" 90% CI : [${lower:.2f}, ${upper:.2f}]
")
```

output

Sample Mean : $65.55k.

90% CI : [$60.65, $70.20k]

(2) Define the sample Data.

→ Create a Numpy array containing 20 loan applicants ~~data~~ incomes (in thousands of dollars).

→ Example : 50 means 50k, '90' means 90k, etc.

(3) Boot strap the Mean.

Example : Cookie

→ Imagine you have a cookie jar with 20 cookies. Each cookie has a number written on it (these are your income numbers : 50, 55, 60, etc).

What we want to know : what's the average number of all cookies in the world (but we only have this one jar).

Here's how bootstrapping works :

1. Take out one cookie, look at its number, and put it back. Do this
   → some cookies might get picked many times.
   → some might not get picked at all.
   → write down all 20 number you picked.

2) Calculate the average of the 20 numbers
   b) write this's average on a separate piece of paper.

(3) Repeat the whole process 1000 times
   → Now you have 1000 average numbers on your paper.

PAGE NO.
DATE / /

4) Sort all these averages from smallest to biggest.

5) Find the edges:
   ↳ The 50th smallest average (this is your low number)
   ↳ The 950th biggest average (this is your high number)

Back to code

bootstrap_means = [np.mean(np.random.choice(data, size=len(data), replace=True)).

(a) → np.random.choice(data, size=len(data), replace=True)
↳ select random number from data by 20 times
original [50, 55, 60 ... 1st 2nd 3rd ... 1
52, 58, 63 ...]
↓ 1st pick ↓ 2nd
↳ [52, ...] → [52, 63, ..] → [52, 63, 50 ..]
→ [52, 63, 50 .... 20th number]
↳ selected number can be repeated by more than once or not at all

(b) → np.mean([52, 63, 50 .... 20th number])
→ Mean [1st mean]

(c) step (a)(b) are repeated to 1000 th times by for _ in range(n_iterations)
→ Mean [1st mean .... 1000th mean]

PAGE NO.....
DATE / /

Pandas.quantile() Vs numpy.percentiles.

numpy's percentiles()

- Works with arrays, calculates the nth percentiles.

Pandas's quantile()

- Works with series / Data Frames, calculate the qth quantile. ($0 \leq q \leq 1$).

Key differences

Feature	numpy.percentiles()	pandas.quantile()
Input Type	Numpy arrays	Pandas series / DataFrames.
Parameter Name	q (0-100 scale)	q (0-1 scale)
Multiple Percentiles	pass as list	Pass as list.
Interpolation	More options	Fewer options.
NaN Handling	Needs np.nanpercentile	Built-in handling
Axis operation	Yes	Yes

DATE / /

Python code

Understanding Point_estimate and margin.

```
import numpy as np.

# Sample data (incomes).
incomes = np.array([50, 55, 60, 65, 70, 45, 80, 75, 55, 90,
                    52, 58, 63, 68, 72, 48, 85, 77, 57, 82])
                    * 1000.

# Calculate components
point_estimate = np.mean(incomes). # 95%
margin = 1.96 * (np.std(incomes) / np.sqrt(len(incomes)))
ci_lower = point_estimate - margin.
ci_upper = point_estimate + margin.

print(f"Point Estimate: ${point_estimate:,.0f}")

print(f"Margin of Error: ±${margin:,.0f}")

print(f"95% CI: [${ci_lower:,.0f}, ${ci_upper:,.0f}]")
```

output

Point Estimate = $65,350.

Margin of Error: ±$5,620.

95% CI : [$59,730, $70,970].

For skewed data (like incomes), report the median with a bootstrap CI instead.

```
From scipy import stats
ci_median = stats.mstats.mquantiles(incomes,
                    prob=[0.025, 0.975]).
```

PAGE NO.....
DATE / /

print(f" Median income 95% CI: [${ci[0]:,.0f}, ${ci[1]:,.0f}]")

output

Median income 95% of CI: [$45,000, $90,000]